Remote Debugging Setup for HyperV Agent

This directory contains scripts and configuration files for setting up remote debugging of the HyperV Agent application on a Windows Server 2025 machine.

Target Environment

- **Remote Host**: 192.168.7.63
- **OS**: Windows Server 2025
- **Credentials**: administrator/Zaq1@wsx
- **Application Port**: 8743

Files Overview

Configuration Files

- `appsettings.remote.json` - Remote-specific application configuration with debug logging enabled
- `launch.json` - Visual Studio Code remote debugging configuration

Scripts

- `setup-winrm.ps1` - Configures WinRM on local machine (run as Administrator)
- `simple-test.ps1` - Tests remote connection
- `deploy-to-remote.ps1` - Deploys and starts the application remotely
- `test-connection.ps1` - Advanced connection testing (has syntax issues, use simple-test.ps1)

Setup Instructions

Step 1: Configure Local Machine WinRM

Run PowerShell as Administrator and execute:

```
cd remote-debug
.\setup-winrm.ps1
```

Step 2: Configure Remote Machine (192.168.7.63)

Log into the remote machine as administrator and run these commands in PowerShell (as Administrator):

```
# Enable PowerShell Remoting
Enable-PSRemoting -Force

# Configure TrustedHosts (allow any host for simplicity)
Set-Item WSMan:\localhost\Client\TrustedHosts -Value '*' -Force
```

```
# Allow unencrypted traffic (for HTTP)
winrm set winrm/config/service '@{AllowUnencrypted="true"}'
winrm set winrm/config/service/auth '@{Basic="true"}'

# Configure firewall
New-NetFirewallRule -DisplayName 'WinRM HTTP' -Direction Inbound -Protocol TCP
-LocalPort 5985 -Action Allow
New-NetFirewallRule -DisplayName 'HyperV Agent HTTP' -Direction Inbound -
Protocol TCP -LocalPort 8743 -Action Allow

# Restart WinRM service
Restart-Service WinRM
```

Step 3: Test Connection

```
.\simple-test.ps1
```

Expected output:

```
Testing connection to 192.168.7.63...
Connection successful!
OS: Microsoft Windows Server 2025 Datacenter
Computer: [COMPUTER-NAME]
.NET Core: Installed
```

Step 4: Deploy and Debug Application

```
# Deploy application to remote server
.\deploy-to-remote.ps1 -Deploy

# Start remote debugging
.\deploy-to-remote.ps1 -Debug

# Or do both in one command
.\deploy-to-remote.ps1 -Deploy -Debug
```

Step 5: Access Application

Once deployed and running:

- **Application**: http://192.168.7.63:8743
- **Swagger UI**: http://192.168.7.63:8743/swagger
- **Health Check**: http://192.168.7.63:8743/api/v1/health

Debugging Commands

Check Application Status

```
.\deploy-to-remote.ps1 -Logs
```

Stop Remote Application

```
.\deploy-to-remote.ps1 -Stop
```

Manual Remote Session

```
$cred = Get-Credential  # Enter administrator/Zaq1@wsx
Enter-PSSession -ComputerName 192.168.7.63 -Credential $cred
```

Visual Studio Code Remote Debugging

1. Copy `launch.json` to your `.vscode` folder
2. Open the project in VS Code
3. Set breakpoints in your code
4. Use F5 to start debugging with "Remote Debug HyperV Agent" configuration

Troubleshooting

Connection Issues

- Verify network connectivity: `ping 192.168.7.63`
- Check WinRM service: `Get-Service WinRM`
- Test WinRM: `Test-WSMan 192.168.7.63`

Application Issues

- Check logs: `.\deploy-to-remote.ps1 -Logs`
- Verify .NET installation on remote machine
- Check Windows Firewall settings
- Ensure Hyper-V role is installed on remote machine

Common Errors

"WinRM cannot process the request"

- Run `setup-winrm.ps1` as Administrator
- Configure remote machine WinRM settings

"Access Denied"

- Verify credentials are correct
- Ensure administrator account is enabled on remote machine

"Connection timeout"

- Check network connectivity
- Verify firewall rules allow WinRM (port 5985) and HTTP (port 8743)

Security Notes

⚠ **Warning**: This configuration uses unencrypted HTTP and basic authentication for simplicity. In production environments:

- Use HTTPS with proper certificates
- Configure Kerberos authentication
- Restrict TrustedHosts to specific machines
- Use domain authentication instead of local accounts

Application Features

The HyperV Agent provides REST API endpoints for:

- Virtual Machine management
- Container operations
- Network configuration
- Storage management
- Health monitoring

All endpoints are documented in the Swagger UI at `/swagger`.